

TALLER 3

1. USO DE APRENDIZAJE NO SUPERVISADO

El aprendizaje no supervisado es una rama fundamental en el campo de la inteligencia artificial y el análisis de datos. Se enfoca en identificar patrones y estructuras ocultas en conjuntos de datos sin la necesidad de etiquetas o supervisión externa. Esto lo hace invaluable en problemas de identificación de patrones, donde se busca comprender la estructura subyacente de los datos y descubrir relaciones inherentes entre las variables.

En el repositorio de talleres <https://github.com/Borreguin/USFQ-Wshop> se proponen algunos datasets, **elegir solamente uno**:

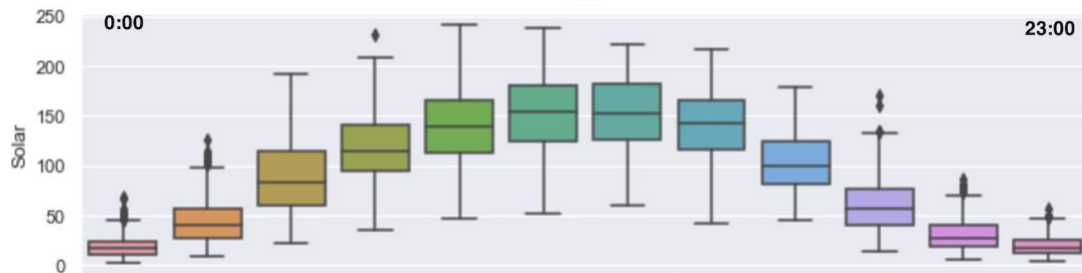
- a) Smart Building Dataset (SBDS) con un set de datos que contiene 4 variables:

Tagname	Alias	Descripción
V005_vent01_CO2	CO2 Ventilation NE	Cantidad de CO2 en ppm de la salida del sistema de ventilación en la parte Norte Este
V022_vent02_CO2	CO2 Ventilation SW	Cantidad de CO2 en ppm de la salida del sistema de ventilación en la parte Sur Oeste
V006_vent01_temp_out	Temp. Vent. NE Out	Temperatura de salida del sistema de ventilación en °C de la parte Norte Este
V023_vent02_temp_out	Temp. Vent. SW Out	Temperatura de salida del sistema de ventilación en °C de la parte Sur Oeste

El set de datos contiene las mediciones de concentración de CO2 y de temperatura del sistema de ventilación de un edificio inteligente. Se considerará como **patrón diario** para este ejercicio, aquel que pertenece al día completo, es decir desde 00:00 hasta 23:59 del día.

- b) Student Productivity & Behavior Dataset (20K) (SP&BDS) se puede descargar el dataset en <https://www.kaggle.com/datasets/algozee/student-productivity-and-behavior-dataset-20k/data>

A. Plotear las variables



Para el caso de SBDS se puede presentar un gráfico por cada variable que muestre sus valores superpuestos por cada día. Para propósito de explicación, la gráfica anterior muestra un ejemplo de una variable de presencia solar que no corresponde a este set de datos, pero sirve de ilustración. Analizar una variable de cada tipo (1 CO2, 1 temperatura) presentar su estadística descriptiva.

Para el caso de SP&BDS se puede usar Coordenadas Paralelas o buscar con PCA cuales son las variables más importantes para plotear. El propósito es encontrar posibles relaciones de manera gráfica. Para motivos del taller 2 variables importantes puede ser suficiente.

B. Encontrar patrones/clústeres – análisis univariable

Utilizando cualquier técnica de aprendizaje no supervisado. Para SBDS encontrar los **patrones diarios** que existen en el data set, para cada variable individual. Para SP&BDS encontrar los clústeres más adecuados, PCA es recomendado para la cantidad de datos a utilizar.

Para SBDS utilizar al menos dos técnicas para verificar su consistencia entre las dos técnicas. Analizar una variable de cada tipo (1 CO2, 1 temperatura), no es necesario las 4.

C. Encontrar anomalías – análisis univariable

Es posible que ciertos perfiles diarios en el set de datos no pertenezcan a los patrones diarios descubiertos en el literal B. ¿Cómo detectarlos? Analizar una variable de cada tipo (1 CO2, 1 temperatura).

D. Encontrar patrones – análisis multivariable

De manera similar al literal B, encontrar los **patrones diarios** que existen en el data set, para cada par de variables, es decir, las dos de la parte Norte Este y/o las dos de la parte Sur Oeste. ¿Utilizar al menos dos técnicas para verificar su consistencia entre las dos técnicas, existe algún patrón el más representativo?

E. Encontrar anomalías – análisis multivariable

De manera similar al literal C, encontrar anomalías, pero de los dos pares de variables.

F. Conclusiones

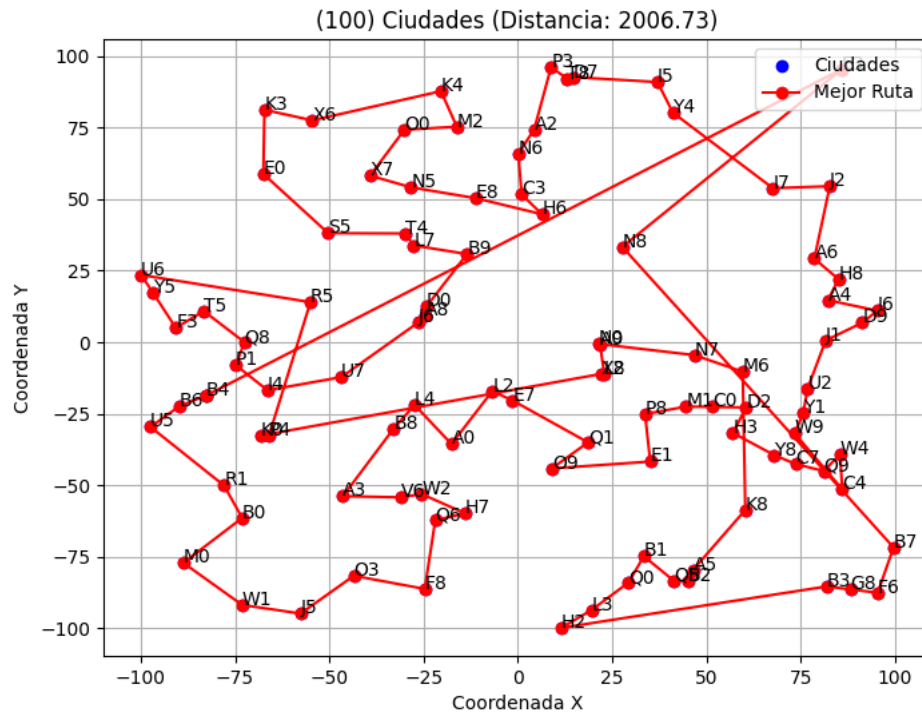
¿Qué pudieran sugerir los patrones y las anomalías encontradas?

2. INVESTIGACIÓN OPERATIVA: TRAVELLING SALEMAN PROBLEM (TSP)

Dentro de la categoría de problemas NP-duro en teoría de la complejidad computacional, el problema del viajante de comercio (TSP) es conocido por ser uno de los problemas clásicos para explicar la complejidad computacional, que puede llegar a tener un problema, dependiendo del número de instancias a resolver. El término NP-duro significa que no existe un algoritmo conocido que pueda resolver todas las instancias del problema en tiempo polinomial. Sin embargo, existen varios enfoques que pueden llegar a obtener soluciones subóptimas en un tiempo razonable.

El algoritmo de vecinos cercanos es una buena opción para obtener una respuesta en tiempo razonable. Por ejemplo, para el problema planteado en el taller 1, con **100 ciudades**, se ha encontrado un camino subóptimo de aproximadamente **2006.73**. ¿Será que usando heurísticas y Linear Programming (LP) se puede alcanzar un recorrido menor?

- Para la ejecución de este problema, instalar el paquete GLPK de acuerdo con lo indicado en el readme.md.
- Se coloca la imagen de la solución del vecino más cercano como referencia, por favor asegurarse que las 100 ciudades correspondan adecuadamente, para el análisis de las siguientes preguntas.



A. Analizar el código propuesto

En la carpeta Taller3/P2_TSP se coloca el código del modelado del TSP usando LP, correr el caso 1, con un mipgap de 0.05 y tiempo límite de 30 segundos, marcar los tiempos que se demora para 10, 20, 30, 40 y 50 ciudades. Comparar los resultados que se puede obtener usando la heurística del vecino cercano (Taller3/P2_TSP/util_nearest_neighbor.py).

Subjetivamente, ¿qué tal te parece las soluciones que ha arrojado el modelo sin aplicar todavía una heurística que ayude al modelo?

B. Analizar el parámetro tee

El parámetro tee se encuentra deshabilitado, activarlo para cualquier interacción, investigar su funcionalidad y comprender su salida en pantalla.

C. Aplicar heurística de límites a la función objetivo

Ejecutar el caso 2, con 70 ciudades. Se ha colocado una heurística que trata de predecir cuanto será el recorrido total. Realizar la corrida de este caso: a) con heurística y b) sin heurística. Explicar los resultados y responder.

1. ¿Cuál es la diferencia entre los dos casos?
2. ¿Sirve esta heurística para cualquier caso? ¿Cuál pudiera ser una razón?

D. Aplicar heurística de vecinos cercanos

Ejecutar el caso 3, con 100 ciudades. Se ha colocado una heurística de vecinos cercanos. Realizar la corrida de este caso: a) con heurística y b) sin heurística. Explicar los resultados y responder.

1. *¿Cuál es la diferencia entre los dos casos?*
2. *¿Sirve esta heurística para cualquier caso? ¿Cuál pudiera ser una razón?*

E. Escribe tus conclusiones

F. (Opcional) Puede ser complicado de implementar

Una de las razones por la que no se alcanza el camino más corto es el cruce que puede haber entre caminos, para ello se puede implementar una heurística que evite los cruces de caminos, investigar como pudiera ser su implementación e implementar esta heurística dentro del modelo.

3. ALGORITMOS GENÉTICOS

El taller contiene el código para la ejecución de un Algoritmo Genético, el objetivo de este algoritmo es llegar a generar la frase objetivo: "GA Workshop! USFQ" que tiene 17 letras. El proceso de generación lo realizará a partir de una población de 100 frases aleatorias (individuos) de tamaño de 17 letras. Las poblaciones generadas deberán interactuar cada una hasta llegar a producir la frase objetivo. Una vez alcanzado un individuo que sea igual a la frase objetivo, el algoritmo genético se detendrá.

Posibles resultados de ejecución:

1. El algoritmo llega a producir el individuo deseado en el número de interacciones que se le ha definido, en este caso, el algoritmo ha convergido a una solución óptima.
2. El algoritmo no logra producir el individuo deseado en el número de interacciones que se le ha definido, en este caso ocurren dos situaciones: a) el algoritmo puede que este cerca del objetivo y le falta más iteraciones para llegar a una solución óptima, b) el algoritmo nunca puede converger y en lugar de acercarse al objetivo se aleja.

En el código entregado, existen dos casos de estudio:

Caso de estudio 1:

DEFAULT (cuenta número de coincidencias para su función de evaluación)

Caso de estudio 2:

DISTANCE (encuentra la diferencia entre el individuo objetivo y el mejor individuo de cada generación)

Estructura del proyecto:

AlgoritmosGeneticos/GA.py:	Programa Principal
AlgoritmosGeneticos/constants.py	Constantes del proyecto
AlgoritmosGeneticos/generalSteps.py	Pasos Generales del Algoritmo Genético
AlgoritmosGeneticos/operation.py	Operaciones dentro de los Pasos Generales
AlgoritmosGeneticos/util.py	Funciones de ayuda

Importante:

Durante las operaciones del AG, **no es permitido** “tomar”, “copiar”, “extraer” información del individuo objetivo, lo que se puede realizar es “leer” y/o “comparar” información del objetivo a fin de comparar el mejor candidato.

Ejecución de los casos:

Para ejecutar los casos, en el archivo *AlgoritmosGeneticos/GA.py* ejecutar la función main.

Ejercicio Propuesto:

1. Ejecute los dos casos de estudio y **explique** los resultados de ejecución de cada caso de estudio.
2. ¿Cuál sería una posible explicación para que el caso 2 no finalice como lo hace el caso 1? Revisar el archivo util.py función distance.
3. Realice una correcta implementación para obtener la distancia/diferencia correcta entre dos individuos en el archivo **util.py** función **distance**. (por ejemplo, considerar alguna métrica que mida mejor la similitud entre dos palabras)
4. ¿Sin alterar el parámetro de mutación **mutation_rate**, se puede implementar algo para mejorar la convergencia y que esta sea más rápida? Implemente cualquier mejora que permita una rápida convergencia. Pista: ¿Tal vez elegir de manera diferente los padres? ¿Realizar otro tipo de mutación o cruce?
5. Cree un **nuevo caso de estudio 3**. Altere el parámetro de mutación **mutation_rate**, ¿ha beneficiado en algo la convergencia? Qué valores son los más adecuados para este parámetro. ¿Qué conclusión se puede obtener de este cambio?
6. Cree un **nuevo caso de estudio 4**. Altere el tamaño de la población, ¿es beneficioso o no aumentar la población?
7. De todo lo aprendido, cree el caso de estudio definitivo (caso de estudio 5) el cual tiene lo mejor de los ítems 4, 5, 6.

